

AI-AUGMENTED HOSPITAL MANAGEMENT SYSTEM

A Queue-First, Cloud-Agnostic Platform for Multi-Hospital Operations

Project Synopsis · Bachelor of Engineering, Computer Science

1. Problem Definition

Conventional hospital management systems are administrative record keepers. They store appointments, bills and prescriptions, but they do not manage the part of the visit a patient actually experiences, which is waiting. A patient in an outpatient department cannot reliably learn how long the wait will be, whether the doctor is present at all, or when their turn is approaching. Reception answers these by hand, all day. The clinical side mirrors it: a doctor meets a patient with no consolidated view of allergies, chronic conditions, current medications or recent results, and reconstructs that context verbally inside a consultation averaging under ten minutes. Three structural gaps follow:

- **Operational.** Queue state is invisible to everyone who depends on it, so waiting rooms are managed by calling out a token number.
- **Clinical.** Patient context is scattered across visits and departments, and is assembled from memory at the moment it matters most.
- **Deployment.** Existing products are either on-premise systems that cannot scale or cloud services that lock patient data into one vendor, which many institutions cannot accept for policy or regulatory reasons.

2. Project Objectives

1. To build a real-time queue keeping the patient's phone, the doctor's workstation and the reception desk on the same state within two seconds of any change.
2. To generate a consolidated one-page *patient sheet* automatically and deliver it to the doctor before the patient enters the room.
3. To integrate consultation, prescription, laboratory, billing and pharmacy into one auditable workflow, removing parallel paper records.
4. To apply AI as a controlled assistant for documentation, report explanation and analytics, under a strict rule that no AI output becomes a clinical record without a clinician's signature.
5. To enforce privacy by design: role-based access control, per-patient ownership checks, complete audit logging, and compliance with the Digital Personal Data Protection Act, 2023.
6. To deliver a cloud-agnostic system running unmodified on a managed cloud, a hospital's own Kubernetes cluster or a single server, so patient data never has to leave the institution.

3. Methodology and System Architecture

The system is a domain-driven microservice platform. Thirteen bounded contexts are consolidated into **eight independently deployable services**, because the latency-critical path (queue to patient sheet to notification) is the one path users watch, and every extra boundary on it adds a network round trip and a failure mode.

Service	Responsibility
gateway	Sole public entry point: JWT verification, identity-header stripping, rate limiting, WebSocket fanout
identity	Users, credentials, roles, sessions, devices, OTP, RS256 token signing
directory	Hospitals, departments, doctors, schedules, attendance, leave, computed availability
scheduling	Appointments, waitlists, queue tokens, priority ordering, consultation state
clinical	Patient records, consultation content, prescriptions, laboratory orders and results, consent
commerce	Billing, invoices, payments, refunds, pharmacy catalogue, inventory, dispensing
comms	In-app, push, SMS, email and WhatsApp delivery behind one notification interface
ai	Agents, per-user memory, typed tool execution, evaluation harness

Communication. Synchronous HTTP is used only where the caller needs a result to respond, such as a slot availability check. Everything else is an asynchronous domain event over RabbitMQ with per-consumer dead-letter queues and idempotent consumers, since at-least-once redelivery is normal operation rather than an error. Live queue updates reach clients over WebSocket, fanned out through Redis so multiple gateway replicas stay consistent, with a polling fallback for unreliable hospital networks.

Data ownership. Each service owns a dedicated PostgreSQL schema and reads no other, giving database-per-service isolation without operating eight clusters. Patient identity is global while every visit-scoped record carries a `hospitalId`, producing true multi-tenancy from the first commit. A unique constraint with sequence-based token generation removes the race that otherwise issues one token to two patients.

AI stance. Clinical facts are never generated; they are retrieved from the owning service at request time through typed, authorization-enforcing tools. The deterministic patient sheet ships first and becomes the measured baseline any AI-generated sheet must beat on allergy and medication recall before it may replace it.

4. Technology Stack

Layer	Technology	Rationale
Language	TypeScript across all tiers	One schema types the API, the web form and the mobile screen
Backend	Node.js 22, Fastify, zod, Prisma	Validation, serialization and typing from one declaration
Data	PostgreSQL 16, Redis, pgvector	Transactional domain; vectors without a second datastore
Messaging	RabbitMQ	Routing, retries, dead-letter queues, delayed delivery
Object storage	MinIO over the S3 API	Private buckets with presigned URLs, portable to any S3 provider
Web	Next.js 15, Tailwind CSS, TanStack Query	One application serving patient, doctor and admin route groups
Mobile	React Native, Expo SDK 54	Over-the-air updates and push without a store review cycle
Desktop	Electron with electron-vite	Printer and scanner access, controlled update on unattended machines
AI layer	LangChain, LangGraph, OpenAI-compatible endpoint	Provider-independent, supports a self-hosted model on request
Payments	Razorpay behind a provider interface	UPI and card coverage, replaceable by configuration
Infrastructure	Docker, Kubernetes, Helm, Terraform	One image, one chart, reproducible provisioning
Observability	Prometheus, Grafana, Loki, OpenTelemetry	Self-hosted, so a customer deployment keeps full visibility

Cloud independence. Every infrastructure dependency sits behind an interface with both a managed-cloud and a self-hosted implementation. Cloud SDKs are confined to one package, enforced by a lint rule that fails the build, and the portable deployment is exercised in continuous integration on every merge. Portability that is not tested is a claim rather than a property.

5. Functional Specifications and Deliverables

- **Patient application (mobile and web).** OTP registration, discovery with live availability, appointment booking, real-time queue tracking with position, estimated wait and near-turn alerts, medical records with patient-controlled consent grants, prescription and invoice downloads, online payment, and an AI copilot restricted to the caller's own records.
- **Doctor workstation (desktop and web).** Attendance, priority-ordered queue control, the automatically delivered patient sheet, structured consultation and SOAP notes, prescribing with allergy and interaction alerts and live stock checks, laboratory ordering with trend comparison, and an AI scribe whose output requires the doctor's signature.
- **Reception and operations desktop.** Walk-in registration, thermal token printing, queue control (skip, recall, merge, reassign) with a full audit trail, the availability board, the billing counter and the dispensing counter, all working through a network outage by queueing writes locally and replaying them.
- **Administration console.** Hospital, department, room, service and fee configuration, staff and role management, patient administration, reporting, and a break-glass workflow making clinical content reachable only with a stated reason, patient notification and a distinct audit record.
- **AI and platform services.** Patient sheet agent, clinical scribe, triage assistant, patient copilot and operations analyst, each with a defined human gate and an audit trail covering the model, the tool calls and whether the output was accepted or edited; plus the API gateway, real-time transport, event bus, five-channel notification service, PDF generation with Devanagari support and a portable deployment across four profiles.

6. Proposed Plan of Work

Incremental delivery with a demonstrable workflow at the end of every phase. Verification is a real clinical run rather than a passing test suite.

Phase	Focus	Verifiable outcome	Duration
P0	Foundation and identity	A patient registers by phone, receives an OTP and authenticates; a forged identity header is rejected	2 wk
P1	The visit loop	A walk-in registered at reception reaches the patient's phone in two seconds and advances everywhere at once	4 wk
P2	Clinical workflow	The patient sheet reaches the doctor first; the consultation produces a signed prescription	3 wk
P3	Commerce and laboratory	Consultation to invoice to payment to dispensing, with stock decremented exactly once	3 wk
P4	AI platform	The AI sheet matches or beats the deterministic baseline on allergy and medication recall	4 wk
P5	Communications, analytics	Every queue change reaches the correct channel per patient preference	2 wk
P6	Hardening	Penetration test closed, 500 concurrent watchers sustained, recovery inside one hour	2 wk
Total, with the deployable release at the end of P3			20 wk

7. Project Scope and Expected Outcome

The system targets multi-speciality hospitals and clinic chains running outpatient departments, where queue management and clinical documentation are the binding operational constraints. It is deliberately not an inpatient system, a telemedicine platform or an insurance claims processor. Success is measured rather than asserted: queue updates within two seconds at the ninety-fifth percentile, the patient sheet on screen before the consultation begins, five hundred concurrent queue watchers per hospital, a one-hour recovery objective, and a full clinic day completed without staff falling back to paper or opening a database client.

8. Group Details

Roll No.	Name of Student	Name of Guide	Approved By (Signature with Date)
01	Aarsh Vaidya	Prof. Abhinay Gudadhe	
03	Abhay Mishra		
22	Aryan Bokde		
28	Bhuvnesh Verma		